Chapter 9. Convolutional Neural Networks

Lectures: Week 5 Mon (Convolutional neural networks: convolution, stride, padding, pooling, equivariance)

Contents

1 Why a New Layer **2**

2 Convolution as a Local Dot Product **2**

2.1 A fixed filter: moving average 2

2.2 Single and multiple channels 2

2.3 Parameter counts 3

3 Stride, Padding and Output Size **3**

4 Learning Filters **4**

5 Shift Equivariance **4**

6 1×1 Convolution, Pooling and Global Average Pooling **4**

7 Algebraic Facts from Past Exams **5**

1 Why a New Layer

An image $X \in \mathbb{R}^{H\times W\times C}$ fed to a fully connected layer is flattened, and $z = A\mathbf{x} + \mathbf{b}$ with $A \in \mathbb{R}^{M\times HWC}$ has $M(HWC+1)$ parameters: for $32\times 32\times 3$ input and $M = 1000$, already 3,073,000. Flattening loses no numbers, but an unrestricted matrix does not encode which pixels are neighbours or that a pattern can appear anywhere. Two assumptions about images motivate convolution:

- **Locality**: nearby pixels jointly form edges, corners and textures, so a unit can look at a small patch (local connectivity).
- **Stationarity**: the same pattern may occur at any position, so the same detector should be used everywhere (weight sharing).

Definition: equivariance and invariance

Let $T_{\boldsymbol{\tau}}$ shift an image, $(T_{\boldsymbol{\tau}}X)_{i,j,c} = X_{i-\tau_1,j-\tau_2,c}$. A feature extractor F is shift *equivariant* if $F(T_{\boldsymbol{\tau}}X) = T_{\boldsymbol{\tau}}F(X)$ (the output moves with the input) and a classifier G is shift *invariant* if $G(T_{\boldsymbol{\tau}}X) = G(X)$ (the output does not change). More generally, for a group acting by $\rho_{\text{in}}(g)$, $\rho_{\text{out}}(g)$, f is equivariant if $f(\rho_{\text{in}}(g)x) = \rho_{\text{out}}(g)f(x)$; invariance is the case $\rho_{\text{out}} = \text{id}$.

2 Convolution as a Local Dot Product

2.1 A fixed filter: moving average

With the 3×3 averaging kernel (1/9 everywhere), $Y_{i,j} = \frac{1}{9}\sum_{u,v=0}^{2} X_{i+u,j+v}$: on an image of 0s and 90s a window with r bright pixels outputs $10r$. Averaging suppresses isolated pixels but blurs edges; a 5×5 average smooths more. A CNN *learns* the kernel; learned weights may be negative and need not sum to one.

2.2 Single and multiple channels

Definition: convolutional layer

With D filters $W \in \mathbb{R}^{D\times K\times K\times C}$, biases $\mathbf{b} \in \mathbb{R}^D$, stride S, padding P,

$$Z_{i,j,d} = b_d + \sum_{u=0}^{K-1}\sum_{v=0}^{K-1}\sum_{c=1}^{C} W_{d,u,v,c}\,\tilde{X}_{Si+u,\,Sj+v,\,c},$$

where $\tilde{X}$ is the padded input. Each output is a dot product of a $K\times K\times C$ patch with a filter spanning *all* input channels, plus a bias. The array $Z_{\cdot,\cdot,d}$ is the feature map of filter d; stacking the D maps gives D output channels. This operation is cross-correlation (no kernel flip); since kernels are learned, the distinction does not matter for expressiveness.

Example: one channel, 2×2 filter

$X = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{pmatrix}$, $W = \begin{pmatrix} 1 & 0 \\ -1 & 2 \end{pmatrix}$, $b = 1$, $S = 1$, $P = 0$: $Z = \begin{pmatrix} 4 & 8 \\ 1 & 1 \end{pmatrix}$. The same four weights and bias are reused at the four positions.

Example: lecture 5×5 example

With the slide's 5×5 image and $W = \begin{pmatrix} 1 & 2 & 3 \\ -4 & 7 & 4 \\ 2 & -5 & 1 \end{pmatrix}$ ($b = 0$): unpadded $Z = \begin{pmatrix} 51 & 66 & 20 \\ 31 & 49 & 101 \\ 15 & 53 & -2 \end{pmatrix}$; with $P = 1$ the output is 5×5 and its central 3×3 block equals the unpadded output (those windows contain the same pixels).

2.3 Parameter counts

One $5 \times 5 \times 3$ filter has 75 weights and one bias; six filters on $32 \times 32 \times 3$ give a $28 \times 28 \times 6$ output with $6(75 + 1) = 456$ parameters. For the same input and output shape:

Layer	Parameters
Fully connected	$4704(3072 + 1) = 14{,}455{,}392$
Locally connected, unshared	$28^2 \cdot 6(75 + 1) = 357{,}504$
Convolution (shared)	$6(75 + 1) = 456$

Locality removes long-range connections; sharing removes independent copies. As a matrix acting on the flattened input, convolution is sparse with repeated entries (a 1D length-2 filter (a, b) on $(x_1, \ldots, x_4)$ gives a banded matrix with only two free numbers).

3 Stride, Padding and Output Size

Theorem: output size

For input length N, kernel K, stride S, padding P (symmetric), the number of output positions is

$$N' = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1, \qquad C' = D.$$

Proof. The padded length is $N + 2P$. A window starting at tS ends at $tS + K - 1$, which must be $\leq N + 2P - 1$, i.e. $t \leq (N + 2P - K)/S$. The admissible integers are $t = 0, \ldots, \lfloor (N + 2P - K)/S \rfloor$. ■

$N = 7$, $K = 3$: $S = 1 \Rightarrow 5$, $S = 2 \Rightarrow 3$, $S = 3 \Rightarrow 2$ (the last does not "fit" exactly; a border is left unused). For $S = 1$ and odd K, $P = (K - 1)/2$ preserves the size (padding $1, 2, 3$ for $K = 3, 5, 7$). Zero padding is an explicit boundary assumption, not recovered information.

Example: lecture examples

Input $52 \times 52 \times 128$, $D = 256$. (1) $K = 3, S = 1, P = 1$: output $52 \times 52 \times 256$, parameters $256(9 \cdot 128 + 1) = 295{,}168$; each filter is $3 \times 3 \times 128$. (2) $K = 4, S = 2, P = 1$: $\lfloor (52 + 2 - 4)/2 \rfloor + 1 = 26$, output $26 \times 26 \times 256$, parameters $256(16 \cdot 128 + 1) = 524{,}544$. Parameters, activations (HWD) and multiply–accumulates ($\approx HWDK^2C$) are different costs.

Stacking: $32 \times 32 \times 3 \to 28 \times 28 \times 6 \to 24 \times 24 \times 10$ with ten $5 \times 5 \times 6$ filters ($10(150 + 1) = 1510$ parameters). Two stacked stride-1 5×5 layers see a 9×9 patch (receptive field $5 + 4$). With strides, $r_\ell = r_{\ell-1} + (K_\ell - 1)j_{\ell-1}$ and $j_\ell = S_\ell j_{\ell-1}$, starting from $r_0 = j_0 = 1$.

4 Learning Filters

For an unpadded stride-1 layer with upstream $G_{i,j,d} = \partial L/\partial Z_{i,j,d}$,

$$\frac{\partial L}{\partial W_{d,u,v,c}} = \sum_{i,j} G_{i,j,d} X_{i+u,j+v,c}, \qquad \frac{\partial L}{\partial b_d} = \sum_{i,j} G_{i,j,d} :$$

every position contributes to the update of the shared weight (the backward counterpart of weight sharing).

5 Shift Equivariance

Theorem: convolution commutes with shifts

On an infinite grid (or with circular indices), stride-1 convolution satisfies $C_W(T_{\boldsymbol{\tau}} X) = T_{\boldsymbol{\tau}} C_W(X)$, and pointwise nonlinearities also commute with shifts. Global average pooling of a circularly shifted map is unchanged, so GAP $\circ$ (convolutions, ReLUs) is shift invariant.

Proof. $[C_W(T_{\boldsymbol{\tau}} X)]_{i,j,d} = b_d + \sum_{u,v,c} W_{d,u,v,c} X_{i+u-\tau_1,j+v-\tau_2,c} = [C_W X]_{i-\tau_1,j-\tau_2,d}$; the coefficients do not depend on the absolute position (i, j). A circular shift permutes positions, and an average is invariant under permutations. ■

Remark: when exactness fails

With zero padding, content shifted across the border is lost and zeros enter, so equality holds only away from the boundary. A stride-S layer (or pooling) is compatible only with shifts that are multiples of S: for width-2, stride-2 max pooling M, $M(T_2 x) = T_1 M(x)$ but $M(T_1 x)$ need not be a shift of $M(x)$ (e.g. $x = (0, 1, 2, 0)$, $M(x) = (1, 2)$, $M(T_1 x) = (0, 2)$).

6 1×1 Convolution, Pooling and Global Average Pooling

1×1 convolution. At each position $\mathbf{y}_{i,j} = A\mathbf{x}_{i,j} + \mathbf{b}$ with $A \in \mathbb{R}^{D \times C}$ shared across positions: it mixes channels, not neighbours. $56 \times 56 \times 64 \to 56 \times 56 \times 32$ costs $32(64 + 1) = 2080$ parameters.

Pooling. Channelwise, no parameters: $Y^{\max}_{i,j,c} = \max_{u,v} X_{iS+u,jS+v,c}$ or the average. For $X = \begin{pmatrix} 5 & 10 & 6 & 7 \\ 4 & 0 & 4 & 3 \\ 2 & 4 & 3 & 4 \\ 0 & 8 & 1 & 6 \end{pmatrix}$ and 2×2, stride 2: average $\begin{pmatrix} 4.75 & 5 \\ 3.5 & 3.5 \end{pmatrix}$, max $\begin{pmatrix} 10 & 7 \\ 8 & 6 \end{pmatrix}$. Pooling reduces resolution and gives limited robustness to small shifts within a window. Backward: max pooling routes the gradient to the maximiser; average pooling spreads g/K^2.

Global average pooling. $g_c = \frac{1}{HW} \sum_{i,j} X_{i,j,c}$ maps $H \times W \times C \to C$; a classifier on top needs $Q(C + 1)$ instead of $Q(HWC + 1)$ parameters. It discards spatial arrangement entirely.

7 Algebraic Facts from Past Exams

Theorem: positive homogeneity (2024 Q6)

A network built only from bias-free convolutions and linear layers, ReLU, max pooling, reshaping and dropout satisfies $f(\lambda X) = \lambda f(X)$ for all $\lambda \geq 0$.

Proof. Each component is positively homogeneous: linear maps for all λ; $\max(0, \lambda t) = \lambda \max(0, t)$ and $\max_i \lambda x_i = \lambda \max_i x_i$ for $\lambda \geq 0$ (scaling by $\lambda > 0$ preserves order and signs; $\lambda = 0$ is trivial); dropout multiplies by a fixed mask. Compositions of positively homogeneous maps are positively homogeneous: $g_1(g_2(\lambda X)) = g_1(\lambda g_2(X)) = \lambda g_1(g_2(X))$. Biases break the property, e.g. $\mathrm{ReLU}(x+1)$ at $x = 0$. ■

Theorem: conv–MP–ReLU orders coincide (2022 Q2)

conv→MaxPool→ReLU, conv→ReLU→MaxPool and conv→ReLU→MaxPool→ReLU are the same function.

Proof. For nondecreasing ϕ, $\max_i \phi(x_i) = \phi(\max_i x_i)$: $\phi(x_i) \leq \phi(\max x)$ for all i with equality at the maximiser. ReLU is nondecreasing and elementwise, and max pooling is a windowwise max, so $\mathrm{ReLU} \circ \mathrm{MP} = \mathrm{MP} \circ \mathrm{ReLU}$; finally $\mathrm{ReLU} \circ \mathrm{ReLU} = \mathrm{ReLU}$. (For average pooling only avg ReLU $\geq$ ReLU avg holds, by convexity.) ■

Theorem: convolutions that implement given maps (2021 Q1, 2022 Q3)

With PyTorch's convention $Y_{o,i,j} = b_o + \sum_c \sum_{k,l=1}^{3} w_{o,c,k,l} \tilde{X}_{c,i+k-2,j+l-2}$ (3×3, padding 1), reading the value at offset $(\Delta i, \Delta j)$ requires a 1 at $(k, l) = (2 + \Delta i, 2 + \Delta j)$.

- Identity $C \to C$: $w_{c,c',k,l} = \delta_{cc'} \delta_{k2} \delta_{l2}$, $\mathbf{b} = \mathbf{0}$.
- Colourwise finite differences $Y_{2c-1} = X_{c,i+1,j} - X_{c,i,j}$, $Y_{2c} = X_{c,i,j+1} - X_{c,i,j}$: $w_{2c-1,c} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$, $w_{2c,c} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 1 \\ 0 & 0 & 0 \end{pmatrix}$, all other slices zero; zero padding supplies the out-of-range zeros.